

# 이론

## [1] 수학 및 통계 기반 (Mathematics & Statistics Foundation)

|               |                                 |
|---------------|---------------------------------|
| 1. 수학 및 통계 기반 | 가설 검정 (p-value, 유의수준)           |
|               | 중심극한정리 (CLT)                    |
|               | 선형대수 (PCA, 고유값/고유벡터)            |
|               | 다중공선성 (VIF, 상관 행렬)              |
|               | 손실 함수 (MSE, MAE, Cross-Entropy) |

### ① 가설 검정 (Hypothesis Testing)

#### 핵심 개념

- 데이터를 통해 모집단에 대한 가설이 참인지 거짓인지 판단하는 통계 기법.
- 귀무가설(Null Hypothesis,  $H_0$ ) vs 대립가설(Alternative Hypothesis,  $H_1$ )
- 검정 통계량(Test Statistic)을 계산하고, 이를 **p-value**와 비교해 유의수준( $\alpha$ ) 이하인지 판단.

#### 용어

- p-value (유의확률)**: 귀무가설이 참일 때, 관측된 결과보다 극단적인 값이 나올 확률.  
→ 작을수록 귀무가설이 맞을 가능성이 낮다.
- 유의수준 (Significance Level,  $\alpha$ )**: 귀무가설을 기각하는 기준값 (보통 0.05 또는 0.01).

#### 시험 포인트

- $p\text{-value} < \alpha \rightarrow$  귀무가설 기각 (통계적으로 유의함).
- $p\text{-value} > \alpha \rightarrow$  귀무가설 채택 (유의하지 않음).
- "귀무가설을 기각한다"  $\neq$  "대립가설이 100% 맞다" ❌

## ② 중심극한정리 (Central Limit Theorem, CLT)

### 핵심 개념

- 모집단의 분포 형태와 무관하게, **\*\*표본 크기(n)\*\***가 충분히 크면 **표본평균의 분포가 정규분포로 수렴**한다.
- 표본평균의 분포:

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

- CLT는 **추정과 가설검정**의 근본 전제.

### 시험 포인트

- 모집단이 비정규분포라도 표본 수가 크면 평균은 정규분포로 근사됨.
- 표본의 크기  $\uparrow \rightarrow$  분산  $\downarrow$  (즉, 표준오차 감소).
- CLT는 표본통계량의 신뢰구간 설정에도 사용됨.

## ③ 선형대수 (Linear Algebra)

### ■ PCA (Principal Component Analysis, 주성분 분석)

- 고차원 데이터를 **분산이 큰 방향으로 축소**해 주요 특징을 추출하는 차원 축소 기법.
- 공분산 행렬의 **\*\*고유값(Eigenvalue)\*\***과 **\*\*고유벡터(Eigenvector)\*\***를 이용해 새로운 축(Principal Components)을 정의.

### 시험 포인트

- 고유값: 각 주성분이 설명하는 분산의 크기.
- 고유벡터: 분산이 가장 큰 방향(새로운 좌표축).
- 첫 번째 주성분이 전체 분산의 대부분을 설명하면 차원 축소 효과 높음.

## ④ 다중공선성 (Multicollinearity)

### 핵심 개념

- 회귀분석에서 **독립변수들 간의 상관관계가 높은 경우**  $\rightarrow$  계수 추정 불안정, 해석 어려움.

### 진단 지표

- VIF (Variance Inflation Factor):**

$$VIF_i = \frac{1}{1 - R_i^2}$$

→ 일반적으로 **VIF > 10**이면 다중공선성 심함.

- \*상관 행렬(Correlation Matrix)\*\*을 통해 변수 간 상관계수 확인 가능.

### 시험 포인트

- 다중공선성 높으면 회귀계수의 분산 커짐 → 예측 신뢰도 저하.
- 해결법: 변수 제거, PCA 적용, 정규화 회귀(Lasso, Ridge) 사용.

## ⑤ 손실 함수 (Loss Function)

### 핵심 개념

- 모델의 예측값과 실제값 간의 오차를 수치화한 함수.  
→ 손실이 작을수록 모델이 잘 학습된 것.

### 종류

| 구분                     | 이름                               | 정의/특징                                      |
|------------------------|----------------------------------|--------------------------------------------|
| 회귀(Regression)         | <b>MSE (Mean Squared Error)</b>  | 제곱 오차 평균. 큰 오차에 민감함.                       |
| 회귀(Regression)         | <b>MAE (Mean Absolute Error)</b> | 절댓값 기준 오차 평균. 이상치(Outlier)에 덜 민감.          |
| 분류<br>(Classification) | <b>Cross-Entropy Loss</b>        | 예측확률과 실제 정답 간의 엔트로피 차이를 측정. 확률 기반 분류에 사용됨. |

### 시험 포인트

- MSE > MAE → 이상치 영향이 큼.
- Cross-Entropy는 **Softmax 출력 + 로그 확률** 형태로 계산됨.
- "손실 함수는 작을수록 모델이 정확하다" → 



## [2] 데이터 핸들링 (Data Handling)

|            |                               |
|------------|-------------------------------|
| 2. 데이터 핸들링 | 데이터 불균형 (Oversampling, SMOTE) |
|            | 결측치 처리 (Imputation)           |
|            | 데이터 분할 (시계열)                  |
|            | 전처리(토크나이저, 불용어)               |
|            | 전처리 (증강, 정규화)                 |

## ① 데이터 불균형 (Imbalanced Data)

### 핵심 개념

- 분류 문제에서 특정 클래스(예: 정상 데이터)가 **대부분을 차지하고**, 소수 클래스(예: 이상 데이터)가 매우 적은 상태.
- 이 경우 모델이 다수 클래스만 맞는 방향으로 **편향(Bias)** 될 수 있다.

### 해결 방법

| 방법                                                        | 설명                                       |
|-----------------------------------------------------------|------------------------------------------|
| <b>Oversampling</b>                                       | 소수 클래스의 데이터를 복제하거나 합성하여 비율을 맞춤.          |
| <b>Undersampling</b>                                      | 다수 클래스 데이터를 일부 제거하여 균형 맞춤.               |
| <b>SMOTE (Synthetic Minority Over-sampling Technique)</b> | 기존 소수 클래스 샘플을 바탕으로 <b>새로운 합성 샘플 생성</b> . |
| <b>Class Weight 조정</b>                                    | 손실함수에서 소수 클래스의 가중치를 높임.                  |

### 시험 포인트

- Oversampling은 과적합(overfitting) 위험 ⚠
- SMOTE는 "데이터 합성 기반" 방법임을 기억.
- F1-score, Recall 등의 지표로 성능 평가해야 함 (정확도만 보면 안 됨).



## SMOTE (Synthetic Minority Over-sampling Technique)



### 개념

- 데이터 불균형(Data Imbalance) 문제를 해결하기 위한 **오버샘플링(oversampling)** 기법.
- 소수 클래스(minority class) 데이터를 단순 복제하는 대신, 새로운 합성 데이터(synthetic samples)를 기존 소수 클래스 데이터의 주변(feature space)에서 **보간(interpolation)** 방식으로 생성한다.



### 작동 원리

1. 소수 클래스 데이터 중 하나를 선택한다.
2. 그 데이터의 **k개의 최근접 이웃(k-nearest neighbors)**을 찾는다.
3. 그 중 무작위로 하나를 골라, 두 점을 잇는 선분 위에서 **랜덤 비율로 보간**하여 새로운 샘플을 만든다.

예:

새 샘플 =  $A + \text{random}(0,1) \times (B - A)$



### 장점

- 단순 복제보다 더 **다양한 데이터 분포**를 만들어 과적합(overfitting)을 줄인다.
- **분류기(classifier)**가 소수 클래스의 결정 경계를 더 잘 학습할 수 있게 한다.



### 단점 / 주의점

- 소수 클래스의 **노이즈 데이터**까지 보강될 수 있어, 오히려 성능이 떨어질 수도 있다.
- **고차원 데이터**나 클래스 경계가 복잡한 경우엔 비현실적인 합성 데이터가 생길 수 있다.
- 따라서 보통 **언더샘플링 기법(예: Tomek Links, ENN 등)**과 함께 사용하기도 함.



### 시험 포인트

| 구분 | 핵심 내용                                                    |
|----|----------------------------------------------------------|
| 목적 | 데이터 불균형 해소 (특히 소수 클래스 보강)                                |
| 방식 | 기존 샘플 주변에서 <b>보간(Interpolation)</b> 으로 합성 데이터 생성         |
| 비교 | 랜덤 오버샘플링(Random Over Sampling)은 단순 복제, SMOTE는 새로운 데이터 생성 |
| 주의 | 노이즈 데이터까지 확장될 위험 존재                                      |
| 응용 | 분류(Classification) 문제에서 주로 사용됨                           |

- **ADASYN(SMOTE 변형 버전)**

: **SMOTE의 변형 기법**으로, 소수 클래스 중에서도 **학습하기 어려운(결정 경계 근처의)** 샘플일수록 **더 많이 합성 데이터**를 만들어준다. 즉, **난이도 기반 적응형 오버샘플링 기법**.

SMOTE는 “**균등 보강**” / ADASYN은 “**어려운 샘플 중심 보강**”

## ② 결측치 처리 (Missing Value Imputation)

### 핵심 개념

- 데이터 중 일부 값이 **비어 있거나 누락된 상태**.
- 결측치가 많으면 분석 결과 왜곡 가능 → 적절히 처리 필요.

### 처리 방법

| 방법                           | 설명                                          |
|------------------------------|---------------------------------------------|
| <b>삭제 (Deletion)</b>         | 결측치 포함 행(row)이나 열(column)을 제거. 단, 데이터 손실 큼. |
| <b>대체 (Imputation)</b>       | 평균(mean), 중앙값(median), 최빈값(mode) 등으로 대체.    |
| <b>예측 기반 대체</b>              | KNN, 회귀모델 등으로 결측값을 예측.                      |
| <b>표본 보간 (Interpolation)</b> | 시계열 데이터의 결측 구간을 이전값·다음값 기준으로 보간.            |

### 시험 포인트

- 단순 대체는 분산을 과소평가할 수 있음.
- 회귀·KNN 기반 대체는 데이터 패턴 유지 가능하지만 연산비용↑.
- 결측치 처리 전, **결측 비율 및 패턴 시각화**가 우선!

### ③ 데이터 분할 (Data Splitting)

#### 핵심 개념

- 모델 학습 시 데이터를 **훈련(train)**, **검증(validation)**, **테스트(test)** 세트로 나누는 과정.
- 일반적인 비율: 6:2:2 또는 7:2:1

#### 방식

| 방법                      | 설명                                      |
|-------------------------|-----------------------------------------|
| Hold-out                | 단순 비율로 한 번 나눔. 가장 기본.                   |
| K-Fold Cross Validation | 데이터를 K개로 나누어, K번 반복 학습 및 검증 → 과적합 방지.   |
| Time Series Split       | 시계열 데이터에서는 시간 순서를 유지해야 함. 과거→미래 순으로 분리. |

#### 시험 포인트

- 랜덤 셔플 시 **데이터 누수(Data Leakage)** 주의.
- 시계열 데이터는 무조건 시간순 분할.
- K-Fold는 데이터가 적을수록 유리.

### ④ 전처리 (Preprocessing)

#### ■ 토큰나이저 (Tokenizer)

- 텍스트 데이터를 단어, 형태소 등 **의미 단위로 나누는 과정**.
- 예:

"나는 학교에 간다" → ["나", "는", "학교", "에", "간다"]

#### 시험 포인트

- 자연어 처리(NLP)에서 가장 첫 단계.
- 토큰나이징 후 **불용어 제거(stopwords removal)**, **정규화(normalization)** 과정 수행.

#### ■ 불용어 제거 (Stopword Removal)

- "은", "는", "이", "가", "the", "is"처럼 **의미 없는 단어 제거**.
- 모델의 노이즈 감소, 연산 효율 개선.

## ■ 정규화 (Normalization)

- 데이터의 스케일(Scale)을 맞춰주는 작업.
- 입력값의 범위를 조정해 모델 학습 안정화.

| 방법                      | 설명                                                      |
|-------------------------|---------------------------------------------------------|
| Min-Max Scaling         | $((x - x_{min}) / (x_{max} - x_{min}))$ , [0,1] 구간으로 조정 |
| Z-score Standardization | $((x - \mu) / \sigma)$ , 평균 0, 표준편차 1로 맞춤               |
| Log Scaling             | 큰 값의 스케일을 줄여 분포 완화                                      |

### 시험 포인트

- 정규화(Normalization) = 0~1 스케일링
- 표준화(Standardization) = 평균 0, 표준편차 1
- PCA, KNN, SVM 등 거리 기반 모델에서는 필수

## ■ 증강 (Augmentation)

- 데이터가 부족할 때 기존 데이터를 변형해 인위적으로 데이터 수를 늘리는 방법.
- 예: 이미지 회전, 반전, 밝기 조정 / 텍스트 교체 / 오디오 노이즈 추가 등

### 시험 포인트

- 딥러닝에서 과적합 방지, 일반화 성능 향상.
- 학습 데이터에만 적용해야 함 (테스트 세트 X).

# AI Modeling

|             |               |                               |
|-------------|---------------|-------------------------------|
| AI Modeling | 1. 핵심 모델링     | ML 학습 방식 (지도, 강화)             |
|             |               | ML 알고리즘 (회귀, 앙상블, SVM, 클러스터링) |
|             |               | 정규화 (L1/L2, 드롭아웃)             |
|             |               | 활성화 함수 (ReLU, Sigmoid)        |
|             |               | 트랜스포머 (어텐션 메커니즘)              |
|             |               | 트랜스포머 (핵심 구조)                 |
|             | 2. 모델 학습 및 평가 | 딥러닝 학습 (역전파, 연쇄 법칙)           |
|             |               | 딥러닝 학습 (기울기 문제)               |
|             |               | 딥러닝 학습 (옵티마이저)                |
|             |               | 모델 검증 (과적합 진단)                |
|             |               | 모델 검증 (데이터 누수)                |
|             |               | 모델 검증 (평가 지표)                 |
|             |               | 모델 검증 (교차 검증)                 |

## 1. 핵심 모델링 (Core Modeling)

### ML 학습 방식

- **지도학습 (Supervised)**: 입력 + 정답(label) → 예측 (예: 회귀, 분류)
- **비지도학습 (Unsupervised)**: 정답 없음 → 패턴 탐색 (예: 클러스터링)
- **강화학습 (Reinforcement)**: 보상 기반의 의사결정 (예: AlphaGo)

💡 시험포인트: 지도 vs 비지도 vs 강화 → “정답 유무, 보상 존재” 기준 구분

### ML 알고리즘

- **회귀(Regression):** 연속값 예측 (예: 집값)
- **분류(Classification):** 범주 예측 (예: 스팸메일 여부)
- **SVM:** 결정 경계로 분리 (서포트 벡터)
- **클러스터링(K-means 등):** 비지도, 유사도 기반 그룹화

💡 포인트: 회귀=연속값 / 분류=범주형 / 클러스터링=비지도



## 1 회귀 (Regression)

- **선형 회귀 (Linear Regression)** : 입력 변수와 출력 변수의 선형 관계 학습.
- **로지스틱 회귀 (Logistic Regression)** : 분류 문제에 사용, 시그모이드 함수를 통해 확률 예측.

## 2 분류 (Classification)

- **SVM (Support Vector Machine)** : 초평면(Hyperplane)으로 클래스 구분, 마진 최대화.
- **Decision Tree (의사결정나무)** : 규칙 기반으로 분기하며 예측.
- **KNN (K-Nearest Neighbors)** : 주변 k개의 데이터를 기준으로 분류.

## 3 클러스터링 (Clustering)

- **K-Means** : 중심점(centroid)을 기준으로 데이터 군집화.
- **DBSCAN** : 밀도 기반 군집화, 이상치(outlier)에 강함.

## 4 앙상블 (Ensemble Methods)

여러 개의 약한 모델(Weak Learner)을 결합해 강한 모델을 만드는 기법.

- **배깅 (Bagging)** :
  - 서로 다른 데이터 샘플로 여러 모델을 학습하고 평균(또는 투표)
  - 대표 알고리즘 → **랜덤 포레스트 (Random Forest)**
  - 장점: 분산(Variance) 감소
- **부스팅 (Boosting)** :
  - 이전 모델의 오차를 보완하는 방향으로 다음 모델을 학습
  - 대표 알고리즘 → **XGBoost, LightGBM, AdaBoost**
  - 장점: 편향(Bias) 감소
- **스태킹 (Stacking)** :
  - 여러 모델의 출력을 **메타 모델(Meta Model)**의 입력으로 사용
  - 예: 1단계에서 여러 알고리즘 예측 → 2단계에서 메타 모델이 최종 예측



## 시험 포인트

- **앙상블의 목적** : 예측 성능 향상 (Bias-Variance Tradeoff 개선)

- **Bagging vs Boosting 비교**

| 구분      | Bagging       | Boosting          |
|---------|---------------|-------------------|
| 학습 방식   | 병렬(parallel)  | 순차(sequential)    |
| 초점      | 분산 감소         | 편향 감소             |
| 대표 알고리즘 | Random Forest | XGBoost, AdaBoost |

## 정규화 (Regularization)

- **L1 (Lasso):** 가중치 절대값 → 일부 가중치 0으로 (특징 선택 효과)
- **L2 (Ridge):** 가중치 제곱 → 과적합 방지
- **드롭아웃 (Dropout):** 뉴런 일부 랜덤 제거 → 과적합 완화

💡 포인트: L1은 가중치 제거, L2는 가중치 축소, Dropout은 뉴런 제거



## 1) 정규화(Regularization) — L1, L2, Dropout, BatchNorm

### 핵심 요약

- **목적:** 모델 복잡도를 제어해 **과적합(overfitting)** 을 줄이고 일반화 성능을 높임.

### L1 (Lasso)

- **직관:** 가중치 절댓값에 페널티를 줘서 일부 가중치를 0으로 만든다 → **특징 선택** 효과.
- **언제 유용한가:** 피처가 많고, 희소성(sparsity)이 기대될 때.
- **시험 포인트:** "L1은 어떤 부수적 이득을 제공하나?" → **희소성 / 변수 제거**.

### L2 (Ridge)

- **직관:** 가중치 제곱에 페널티를 줘 전체 가중치를 작게 만든다 → 계수 안정화.
- **언제 유용한가:** 다중공선성 문제 완화 / 과도한 가중치 감소.
- **시험 포인트:** "L2는 왜 과적합을 줄이나?" → 큰 가중치에 큰 패널티 부과.

### Dropout

- **직관:** 학습 시 무작위로 일부 뉴런(bit)을 끄 → 여러 다른 서브네트워크를 학습하는 셈 → 앙상블 효과.
- **언제 유용한가:** 큰 네트워크에서 과적합 우려가 있을 때.
- **주의:** 추론시(dropout off)엔 가중치 보정이 필요(프레임워크가 자동 처리).
- **시험 포인트:** "Dropout은 훈련/추론 시 어떻게 다른가?" → 훈련 시만 적용, 추론 시 비활성.

### Batch Normalization (BatchNorm)

- **직관:** 각 배치 단위로 활성화의 평균·분산을 정규화 → 내부 공변량 변화 (Internal Covariate Shift) 완화 → 학습 안정화·속도 개선.
- **부수 효과:** 학습률을 크게 할 수 있게 돕고, 가끔은 Dropout과 대체되기도 함.
- **시험 포인트:** "BatchNorm의 목적과 추론 시 동작 차이" → 배치 통계 사용, 추론시엔 축적된 평균·분산 사용.

## 📖 활성화 함수 (Activation Function)

- **Sigmoid**: 0~1 사이 출력, 하지만 기울기 소실(vanishing) 문제 있음
- **ReLU**: 음수는 0, 양수는 그대로 → 학습 빠름, 현재 기본값
- **Leaky ReLU / GELU**: ReLU 단점 보완

💡 포인트: ReLU = 기본, Sigmoid = 확률 표현, Tanh = -1~1 범위

## 💡 2) 활성화함수(Activation Functions) — ReLU, Sigmoid, Tanh, Leaky, GELU

### 핵심 요약

- **목적**: 선형층을 비선형으로 만들어 신경망이 복잡한 함수 근사 가능케 함.

### Sigmoid

- **직관**: 출력값을 0~1로 압축(확률 해석 가능).
- **단점**: 큰 입력에서 기울기 소실(vanishing gradient) → 깊은 네트워크 학습 어려움.
- **시험 포인트**: "Sigmoid의 기울기 소실 문제" — 출력 포화 영역에서 미분 거의 0.

### Tanh

- **직관**: -1~1 범위, 평균 0에 가까움 → Sigmoid보다 학습이 조금 더 빠름.
- **단점**: 여전히 포화 영역 존재.

### ReLU (Rectified Linear Unit)

- **직관**: 음수는 0, 양수는 선형 → 계산 단순·수렴 빠름.
- **단점**: 음수 영역에서 gradient=0 → "Dying ReLU(죽은 뉴런)" 문제.
- **시험 포인트**: "ReLU의 장단점" — 간단·빠르나 음수 영역 문제 존재.

### Leaky ReLU / ELU / GELU

- **직관**: 음수 영역에도 작은 기울기 허용 → Dying ReLU 완화.
- **시험 포인트**: "왜 Leaky ReLU를 쓰는가?" → 음수 구간 기울기 유지.

## 📖 트랜스포머 (Transformer)

- 어텐션 메커니즘 (Attention Mechanism): 입력 중 중요한 부분에 가중치 집중
- 핵심 구조: 인코더-디코더, 멀티헤드 어텐션, 피드포워드, 레이어 정규화

💡 포인트: "순차 모델(RNN)" 대신 "병렬적 처리 + 어텐션 집중"

### 💡 4) 트랜스포머(Transformer) — Attention 핵심 직관

#### 핵심 요약

- 목적: 순서가 있는 입력(문장 등)을 병렬로 처리하면서도 단어간 관계(어텐션)를 학습.

#### Attention (Self-Attention)

- 직관: 각 토큰이 문장 내 다른 토큰들과 얼마나 관련 있는지(가중치)를 계산해, 중요한 토큰의 정보를 동적으로 집계한다.
- 효과: 장거리 의존성(long-range dependency)을 잘 잡음(예: 먼 위치의 단어도 직접 참조 가능).
- 시험 포인트: "Attention의 장점 vs RNN" → 병렬 처리 가능 + 장거리 의존성 포착.

#### Multi-Head Attention

- 직관: 서로 다른 하위공간(subspace)에서 여러 개의 어텐션을 병렬로 수행 → 다양한 관계 포착.
- 시험 포인트: "멀티헤드의 의미" = 다양한 패턴을 동시에 학습.

#### Positional Encoding

- 직관: Attention은 위치정보가 없으므로 위치(순서)를 인코딩으로 추가.
- 시험 포인트: "왜 포지셔널 인코딩이 필요한가?" → 순서 정보 부여.



## 1 핵심 개념: Attention Mechanism

### ◆ 목적

- 입력 토큰들 간의 **의존 관계를 학습**해서, 문맥(context)을 반영한 표현을 얻기 위함.
- 즉, "이 단어를 이해하려면 문장 내 다른 어떤 단어에 주목해야 하는가?"를 계산함.

### ◆ 구조: Scaled Dot-Product Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- **Q (Query):** 현재 단어의 "질문" — 어떤 정보를 찾고 싶은가
- **K (Key):** 문장 내 각 단어의 "키워드" — 어떤 정보를 가지고 있는가
- **V (Value):** 그 단어가 가진 "실제 정보"

### 흐름

1. Q와 K의 내적 → 연관도 점수(score) 계산
2.  $\sqrt{d_k}$ 로 스케일 조정 (값 폭발 방지)
3. Softmax로 정규화 → 가중치화
4. 가중치를 V에 곱해 → "문맥 반영된 벡터" 산출

### ◆ Multi-Head Attention

- 위의 Attention을 여러 개 병렬로 수행 (각 Head마다 Q, K, V projection이 다름)
- 다양한 시각에서 관계를 학습
- Head들의 출력을 concat → Linear projection

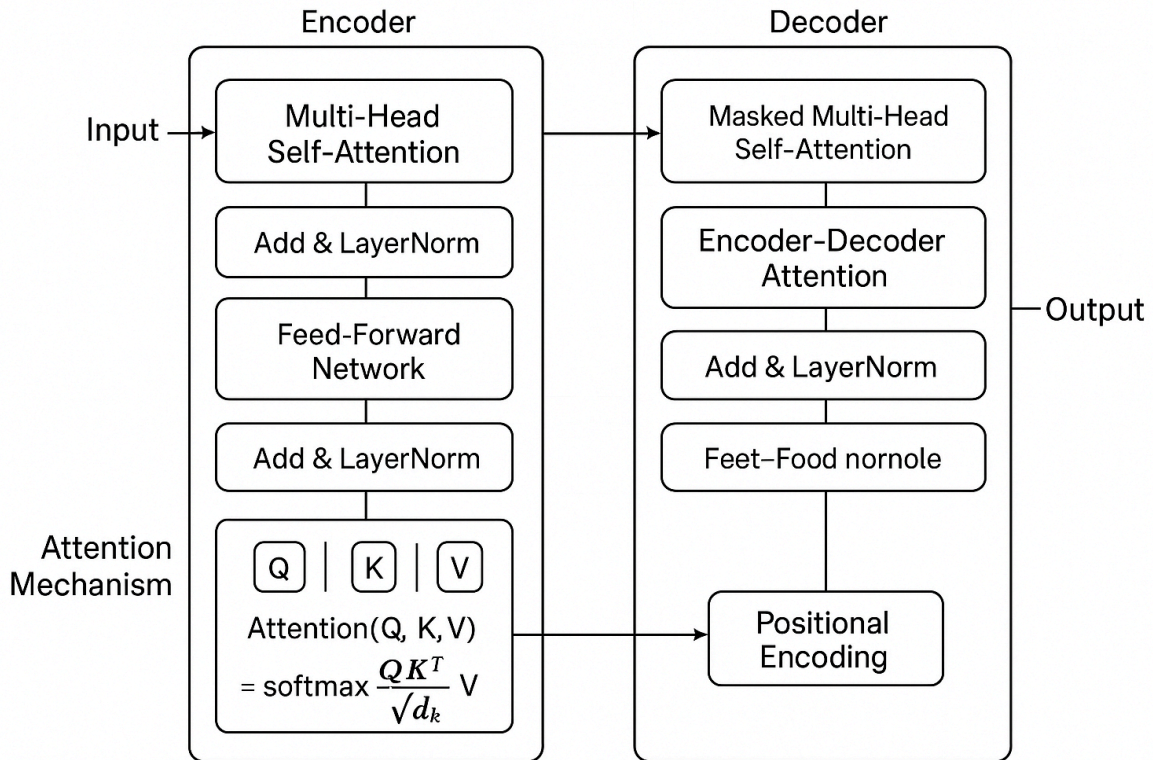
즉,

| "문맥을 여러 관점(문법, 의미, 위치 등)에서 동시에 본다."



## 2 트랜스포머의 핵심 구조 (Architecture)

### 전체 구성



트랜스포머는 **Encoder**와 **Decoder**로 이루어짐.

입력 → [Encoder × N] → 중간 표현 → [Decoder × N] → 출력

### ◆ Encoder Block (N층 반복)

각 층 구성:

#### 1. Multi-Head Self-Attention

- 입력 내 단어들끼리 관계를 계산

#### 2. Add & LayerNorm

#### 3. Feed-Forward Network (FFN)

- 2층 MLP:  $\text{ReLU}(W1x + b1) \rightarrow W2x + b2$

#### 4. Add & LayerNorm

입력 문장을 문맥적으로 잘 표현된 임베딩으로 변환함.

### ◆ Decoder Block (N층 반복)

### 1. Masked Multi-Head Self-Attention

- 현재 시점 이후의 단어는 못 보게 마스킹

### 2. Encoder-Decoder Attention

- Encoder 출력과 상호작용 → 입력 문맥 참고

### 3. Feed-Forward + Add & Norm

이전까지의 생성 토큰 + 입력 문맥을 이용해 다음 단어 예측.

## ◆ 추가 요소

#### • Positional Encoding

순서 정보가 없으므로, 위치 정보를 `sin/cos` 함수를 통해 벡터에 더함.

#### • Residual Connection

각 서브모듈 출력 + 입력을 더함 → 안정적 학습 가능.

#### • Layer Normalization

학습 안정화, 수렴 속도 개선.

## 📖 요약 정리표

| 구분        | 역할         | 주요 구성요소                                   | 핵심 수식 / 개념                         |
|-----------|------------|-------------------------------------------|------------------------------------|
| Attention | 단어 간 관계 계산 | Q, K, V / Scaled Dot-Product / Multi-Head | $\text{Softmax}(QK^T/\sqrt{d_k})V$ |
| Encoder   | 입력 문맥 이해   | Self-Attn + FFN                           | Residual + LayerNorm               |
| Decoder   | 단어 생성      | Masked Self-Attn + Encoder-Attn + FFN     | autoregressive masking             |
| 기타        | 보조 구조      | Positional Encoding / Normalization       | sin, cos 주기함수 기반                   |

## 🧠 2. 모델 학습 및 평가 (Model Training & Evaluation)

|               |                     |
|---------------|---------------------|
| 2. 모델 학습 및 평가 | 딥러닝 학습 (역전파, 연쇄 법칙) |
|               | 딥러닝 학습 (기울기 문제)     |
|               | 딥러닝 학습 (옵티마이저)      |
|               | 모델 검증 (과적합 진단)      |
|               | 모델 검증 (데이터 누수)      |
|               | 모델 검증 (평가 지표)       |
|               | 모델 검증 (교차 검증)       |

## ✓ 핵심 요약

| 개념     | 핵심 키워드               | 시험 포인트                  |
|--------|----------------------|-------------------------|
| 역전파    | 오차 역전달, Chain Rule   | 경사하강법과 함께 출제            |
| 기울기 소실 | Gradient $\approx$ 0 | ReLU, BN, ResNet        |
| 옵티마이저  | Adam, SGD            | 학습률 자동 조정               |
| 과적합    | 학습데이터 과몰입            | Dropout, Early Stopping |
| 데이터 누수 | test 정보 유입           | 전처리 시점 주의               |
| 평가 지표  | F1-score, AUC        | 불균형 데이터 주의              |
| 교차 검증  | K-Fold               | 소량 데이터 일반화              |

## 1 딥러닝 학습 원리

### ■ 역전파 (Backpropagation)

- 오차(손실)를 출력층에서 입력층으로 역전파하여 가중치를 업데이트하는 알고리즘.
- 핵심은 **Chain Rule(연쇄 법칙)** 을 이용해 미분값을 계산한다는 것. **기울기(gradient)** 계산을 통한 **경사하강법**
- 목적: **손실함수 값을 최소화**하도록 각 가중치를 조정.
- 💡 **시험 포인트:**
  - 경사하강법(Gradient Descent)과 함께 자주 묶여서 나옴.
  - "미분을 통해 가중치 업데이트" 라는 문장에 주목. "오차 역전파" = 가중치 업데이트 과정

---

## ▪ 연쇄 법칙 (Chain Rule)

- 여러 함수가 합성된 형태의 미분을 구할 때,  
각 함수의 미분값을 **곱으로 연결**해 전체 미분을 계산.

예:  $f(g(x)) \rightarrow f'(g(x)) \times g'(x)$

---

## ▪ 기울기 소실 문제 (Vanishing Gradient Problem)

- 역전파 중 **기울기(Gradient)** 가 0에 가까워져, 앞쪽 레이어로 전달되지 못하는 현상.
- **Sigmoid, Tanh** 같은 함수에서 자주 발생.
- 해결 방법:

- **ReLU** 계열 활성화함수 사용
- **Batch Normalization** 적용
- **Residual Connection (ResNet)** 도입

- 💡 **시험 포인트:**

“깊은 신경망일수록 학습이 어려운 이유”로 자주 출제됨. 깊은 신경망에서 학습이 멈춤 → 원인: 활성화함수

---

## ▪ 옵티마이저 (Optimizer)

손실함수를 최소화하기 위해 가중치를 갱신하는 규칙(알고리즘)

- **SGD (Stochastic Gradient Descent)** : 기본형, 경사하강, 미니배치 단위로 학습.
- **Momentum** : 이전 방향(속도)을 고려해 진동 완화. 관성 개념.
- **Adam** : Adaptive + Momentum — 적응형 학습률. 가장 널리 쓰임.

- 💡 **시험 포인트:**

Adam이 SGD보다 빠르고 안정적인 이유 → **학습률을 파라미터별로 자동 조정**하기 때문.  
Adam = 실무 표준 / Momentum = 진동 억제

---



### 3) 옵티마이저(Optimizers) — SGD, Momentum, Adam, RMSProp

#### 핵심 요약

- 목적: 손실함수의 기울기(gradient)를 이용해 파라미터를 효율적으로 갱신.

#### SGD (Stochastic Gradient Descent)

- 직관: 작은 배치 단위로 경사하강. 단순하지만 안정적.
- 시험 포인트: "SGD의 한계" → 진동, 느린 수렴(특히 좁고 가파른 지역).

#### Momentum

- 직관: 관성 개념을 도입해 이전 기울기를 일부 반영 → 진동 완화·수렴 가속.
- 시험 포인트: "Momentum의 역할" = 관성, 진동 완화.

#### RMSProp

- 직관: 최근 기울기의 제곱 평균으로 학습률을 조절 → 좌표별 적응적 스케일.
- 시험 포인트: "왜 RMSProp은 학습률 조정에 유리한가?" → 기울기 크기 기반 스케일링.

#### Adam

- 직관: Momentum(1차 모멘트) + RMSProp(2차 모멘트) 결합 → 빠른 수렴, 실무 표준.
- 주의: 일반화(특히 최종 성능)에서 SGD보다 나쁠 때가 있다(일부 연구).
- 시험 포인트: "Adam의 장단점" — 빠른 탐색·편리하지만 때로 일반화가 아쉽음.

### 과적합 진단 (Overfitting)

- 훈련 정확도↑ / 검증 정확도↓ → 과적합 신호
- 해결법: Dropout, 정규화, 데이터 확장



포인트: Train-Validation Gap이 크면 Overfitting



## 5) 과적합 진단 & 해결 (Overfitting Detection & Remedies)

### 진단 신호

- 훈련 성능 ↑ / 검증 성능 ↓ (train-val gap 큼) → 과적합 의심.
- 검증 손실이 증가하는데 훈련 손실은 감소하면 과적합.

### 해결책(우선순위)

1. 데이터 확장(Data Augmentation) — 더 많은 다양성 제공
2. 정규화(L1/L2) — 가중치 제약
3. Dropout — 뉴런 랜덤 제거
4. Early Stopping — 검증 성능이 더 이상 개선되지 않으면 중단
5. 간단한 모델 사용 — 파라미터 수 줄이기

### 시험 포인트

- “Train loss는 줄고 Val loss는 늘 때” → Early Stopping 또는 정규화 필요.
- “과적합 vs 과소적합” 구분 문제 자주 출제됨.



## 데이터 누수 (Data Leakage)

- 검증 데이터가 학습 중에 섞인 경우
- 실제보다 성능이 과대평가됨



포인트: “평가 데이터가 훈련 중 쓰이면 안 된다”



## 7) 데이터 누수(Data Leakage) — 치명적 함정

### 핵심

- 학습 시점에 검증/테스트 정보를 잘못 사용하면 성능이 부풀려짐 → 실제 서비스에서는 실패.
- 예: 스케일러를 전체 데이터에 fit 후 분할 → 누수 발생.

### 방지법

- 전처리(스케일링, 임putation 등)는 **train에 fit → train/val/test에 transform**만 적용.
- 피처 생성시 미래 정보 사용 금지(특히 시계열).

### 시험 포인트

- “어떤 전처리가 데이터 누수를 유발하는가?” → 전체 데이터 기준으로 fit 하는 모든 작업.



## 평가 지표 (Evaluation Metrics)

- 회귀: MSE, RMSE, MAE
- 분류: Accuracy, Precision, Recall, F1
- 불균형 데이터 → F1, ROC-AUC 선호

| 💡 포인트: Accuracy는 불균형 데이터에 취약



## 6) 평가 지표(Evaluation) — 핵심 감별 포인트

### 분류

- **Accuracy:** 전체 비율, 불균형 데이터에서는 오해의 소지.
- **Precision / Recall:**
  - Precision =  $TP / (TP + FP)$  — 긍정 예측의 정확도
  - Recall =  $TP / (TP + FN)$  — 실제 긍정 중 얼마나 찾았나
- **F1 Score:** Precision과 Recall의 조화평균 — 불균형 데이터에 유리
- **ROC-AUC vs PR-AUC:**
  - ROC-AUC는 전체적 분류성능(양성/음성 균형)
  - PR-AUC는 양성 희소 상황에서 더 의미 있음

### 회귀

- **MSE / RMSE / MAE** — 이상치 민감도 및 단위 차이 고려.

### 시험 포인트

- “불균형 데이터에서는 어떤 지표?” → F1 또는 PR-AUC.
- “Precision 높이면 Recall은?” → 일반적으로 trade-off (둘 다 동시에 높이기 어려움).

## 모델 평가 지표 (Evaluation Metrics)

모델이 얼마나 잘 작동하는지를 수치로 표현한 것.

데이터의 특성과 \*\*문제 유형(분류/회귀/랭킹 등)\*\*에 따라 적절한 지표를 골라야 함.

### ✅ (1) 분류(Classification) 문제

#### ▪ 정확도 (Accuracy)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- 전체 샘플 중 맞게 예측한 비율.

-  **클래스 불균형**이 심한 경우에는 부적절 (예: 99%가 정상인데 "전부 정상"이라 해도 99% 정확도).

---

## ▪ 정밀도 (Precision)

$$Precision = \frac{TP}{TP + FP}$$

- "양성이라고 예측한 것 중 진짜 양성 비율"
- 예: 스팸메일 분류 → 스팸이라 한 것 중 실제 스팸 비율.

---

## ▪ 재현율 (Recall)

$$Recall = \frac{TP}{TP + FN}$$

이 수식은 "진짜 양성(True Positive) 중에서 모델이 올바르게 예측한 비율"을 나타냅니다.

- "진짜 양성 중에서 모델이 잘 잡은 비율"
- 예: 암 진단 모델 → 암 환자 중 놓치지 않고 잘 잡은 비율.

---

## ▪ F1 Score

$$[F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}]$$

- 정밀도와 재현율의 **조화 평균**.
- 불균형 데이터셋에서 **균형 성능 측정**에 유용.

---

## ▪ ROC-AUC (Area Under the Curve)

- 분류 임계값을 변화시키며 **\*\*TPR(재현율)\*\***과 **\*\*FPR(거짓 양성 비율)\*\***의 관계를 시각화.
- AUC 값이 1에 가까울수록 모델의 구분 능력이 좋음.

---

## (2) 회귀(Regression) 문제

## ▪ MAE (Mean Absolute Error)

$$[MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|]$$

- 오차 절대값의 평균.
- 실제 단위 그대로 해석 가능 (예: "예측이 평균적으로 2.3달러 틀렸다").

## ▪ MSE (Mean Squared Error)

$$[MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2]$$

- 큰 오차에 더 민감.
- 미분이 쉬워서 **경사하강법**에서 자주 사용.

## ▪ RMSE (Root MSE)

$$[RMSE = \sqrt{MSE}]$$

- MSE의 제곱근 → 실제 단위와 동일.
- 직관적으로 해석 가능.

## ▪ R<sup>2</sup> (결정계수)

$$[R^2 = 1 - \frac{SS_{res}}{SS_{tot}}]$$

- 모델이 전체 변동성 중 얼마나 설명했는지 나타냄.
- 1에 가까울수록 좋고, 0 이하면 모델이 평균보다 못한 것.

---

## ✅ (3) 랭킹 / 추천 (Ranking / Recommendation)

- **MAP (Mean Average Precision)**: 추천 순서 내 정확도 평가.
- **NDCG (Normalized Discounted Cumulative Gain)**: 추천 순서의 중요도 반영.
- **Hit Ratio / Recall@K**: 상위 K개 안에 정답이 있는지.

---

## 📖 교차 검증 (Cross Validation)

- 데이터를 여러 Fold로 나눠서 반복 학습 → 과적합 방지
- **K-Fold, Stratified K-Fold(라벨 비율 유지)**

💡 포인트: "데이터 부족 시 안정적인 성능 평가 방법"X



## 2 교차검증 (Cross Validation)

단순히 train/test split으로 평가하면 데이터 분포나 샘플링 운에 따라 결과가 달라질 수 있음.

이를 방지하기 위한 일반화 성능 검증 기법.

### ✓ (1) K-Fold Cross Validation

- 데이터를 K개로 나눠서,  
매번 하나는 검증용으로, 나머지 K-1개는 학습용으로 사용.
- K번 반복 → 평균 성능으로 최종 평가.
- 일반적으로 **K=5** 또는 **K=10** 사용.

```
Fold1: [Test][Train][Train][Train][Train]
Fold2: [Train][Test][Train][Train][Train]
...
Fold5: [Train][Train][Train][Train][Test]
```

### ✓ (2) Stratified K-Fold

- 클래스 불균형 데이터에서  
각 폴드마다 클래스 비율을 원본 데이터와 동일하게 유지.
- 분류 문제에서 거의 필수.

### ✓ (3) Leave-One-Out (LOOCV)

- 샘플이 적을 때 사용.
- 한 개 샘플만 검증용으로, 나머지는 학습용으로 사용.
- 정확하지만 계산량 매우 많음.

### ✓ (4) Time Series Split

- 시계열 데이터의 경우 과거 → 미래 순서를 보존해야 하므로  
무작위 분할 대신 **시간 순서대로 분리**.
- 예:

```
Train: Jan~Mar → Test: Apr
Train: Jan~Apr → Test: May
```

Train: Jan~May → Test: Jun

## ✓ (5) Nested Cross Validation

- 하이퍼파라미터 튜닝 시 과적합을 방지하기 위해  
\*\*안쪽(inner)\*\*에서는 튜닝, \*\*바깥쪽(outer)\*\*에서는 평가 수행.

## (정리 요약표)

| 구분    | 대표 지표                     | 특징          | 주의점        |
|-------|---------------------------|-------------|------------|
| 분류    | Accuracy, F1, ROC-AUC     | 성능 직관적      | 불균형 데이터 주의 |
| 회귀    | MSE, RMSE, $R^2$          | 오차 크기 해석 가능 | 이상치에 민감    |
| 랭킹/추천 | NDCG, MAP                 | 순서 중요성 반영   | 복잡한 계산     |
| 교차검증  | K-Fold, Stratified K-Fold | 일반화 성능 평가   | 계산비용 있음    |

## AI Engineering

|                |                |                         |
|----------------|----------------|-------------------------|
| AI Engineering | 1. 모델 최적화 및 서빙 | 모델 경량화 (주요 기법)          |
|                | 2. 시스템 및 인프라   | 분산 처리 (대규모 학습)          |
|                | 3. 개발 도구 및 환경  | 형상 관리 (Git 명령어 및 충돌 해결) |
|                |                | AI 프레임워크 (PyTorch)      |



## 요약 정리표

| 구분     | 핵심 개념                               | 키워드 |
|--------|-------------------------------------|-----|
| 모델 최적화 | Pruning, Quantization, Distillation | 경량화 |

| 구분      | 핵심 개념                         | 키워드     |
|---------|-------------------------------|---------|
| 시스템/인프라 | Data Parallel, Model Parallel | 대규모 학습  |
| 형상관리    | git pull, merge, conflict 해결  | 협업      |
| 프레임워크   | Tensor, Autograd, Optimizer   | PyTorch |

## 1 모델 최적화 및 서빙

### ◆ 모델 경량화 (Model Compression)

모델의 메모리 사용량과 연산량을 줄여

실시간 서비스나 엣지(Edge) 디바이스에서 빠르게 동작하도록 하는 기술.

#### 💡 주요 기법

| 기법                                      | 설명                                        | 예시 / 키포인트                   |
|-----------------------------------------|-------------------------------------------|-----------------------------|
| <b>Pruning (가지치기)</b>                   | 중요하지 않은 가중치 (Weight)를 제거                  | 작은 가중치 → 0으로 만들 → 모델 사이즈 감소 |
| <b>Quantization (양자화)</b>               | 가중치 값을 float32 → int8 등 저정밀도로 변환          | 성능 약간 감소하지만 계산속도↑           |
| <b>Knowledge Distillation (지식 증류)</b>   | 큰 모델(Teacher)의 출력을 이용해 작은 모델(Student)을 학습 | 모델 경량화 + 성능 유지              |
| <b>Weight Sharing / Parameter Tying</b> | 유사한 가중치 그룹화                               | 모바일 모델에서 주로 사용              |
| <b>Tensor Decomposition</b>             | 행렬을 저차원 형태로 분해                            | SVD, CP decomposition 등 활용  |

#### ✅ 시험 포인트

- "경량화 기법 중 Teacher-Student 구조를 사용하는 것은?" → *Knowledge Distillation*
- "정밀도(Precision)를 줄여 연산량을 감소시키는 것은?" → *Quantization*

## 2 시스템 및 인프라

## ◆ 분산 처리 (Distributed Training)

대규모 모델을 여러 GPU/서버에서 병렬로 학습시키는 방법.  
메모리 한계를 극복하고 학습 속도 향상.

### 💡 주요 방식

| 방식                          | 설명                          | 대표 프레임워크                                                                  |
|-----------------------------|-----------------------------|---------------------------------------------------------------------------|
| <b>Data Parallelism</b>     | 데이터를 분할해 여러 GPU에서 같은 모델을 학습 | <code>torch.nn.DataParallel</code> , <code>DistributedDataParallel</code> |
| <b>Model Parallelism</b>    | 모델을 여러 GPU에 분할 (매우 큰 모델용)   | Megatron-LM, DeepSpeed                                                    |
| <b>Pipeline Parallelism</b> | 모델 레이어를 나눠 순차적으로 학습         | Transformer 대형 모델 학습                                                      |
| <b>Hybrid Parallelism</b>   | 위의 방법을 조합                   | GPT, LLaMA 등 초거대 모델                                                       |

## ◆ 분산 학습 시 유의사항

- 동기(Synchronous) vs 비동기(Asynchronous) 업데이트 구분
- **Gradient Accumulation**으로 GPU 메모리 절약 가능
- **Mixed Precision Training (FP16/FP8)** → 속도 향상, 메모리 절약

### ✅ 시험 포인트

- "데이터를 나누어 동일한 모델을 병렬로 학습하는 방식은?" → *Data Parallelism*
- "모델이 너무 커서 여러 GPU에 나누는 방법은?" → *Model Parallelism*

## 3 개발 도구 및 환경

### ◆ 형상 관리 (Version Control, Git)

협업 및 코드 버전 추적에 사용되는 기본 도구.

## 💡 주요 명령어

| 명령어                                    | 설명           |
|----------------------------------------|--------------|
| <code>git clone &lt;repo&gt;</code>    | 원격 저장소 복제    |
| <code>git branch &lt;name&gt;</code>   | 새 브랜치 생성     |
| <code>git checkout &lt;name&gt;</code> | 브랜치 이동       |
| <code>git merge &lt;branch&gt;</code>  | 브랜치 병합       |
| <code>git pull</code>                  | 원격 변경사항 가져오기 |
| <code>git push</code>                  | 로컬 변경사항 업로드  |
| <code>git log</code>                   | 커밋 이력 확인     |

## 🚀 Git 대표 명령어 정리

### 🧩 1 기본 환경 설정

| 목적         | 명령어                                                            | 설명            |
|------------|----------------------------------------------------------------|---------------|
| 사용자 이름 설정  | <code>git config --global user.name "홍길동"</code>               | 전역 사용자 이름 설정  |
| 사용자 이메일 설정 | <code>git config --global user.email "hong@example.com"</code> | 전역 이메일 설정     |
| 설정 확인      | <code>git config --list</code>                                 | 현재 설정값 확인     |
| 도움말 보기     | <code>git help &lt;명령&gt;</code>                               | 특정 명령어 도움말 보기 |

### 📁 2 저장소 초기화 및 클론

| 목적           | 명령어                                                     | 설명                  |
|--------------|---------------------------------------------------------|---------------------|
| 새 Git 저장소 생성 | <code>git init</code>                                   | 현재 폴더를 Git 저장소로 초기화 |
| 원격 저장소 복제    | <code>git clone &lt;URL&gt;</code>                      | GitHub 등 원격 저장소를 복제 |
| 서브모듈 포함 복제   | <code>git clone --recurse-submodules &lt;URL&gt;</code> | 하위 모듈 포함 복제         |

### 🔄 3 변경 추적 및 스테이징

| 목적       | 명령어                     | 설명                 |
|----------|-------------------------|--------------------|
| 상태 확인    | <code>git status</code> | 변경된 파일, 스테이징 상태 확인 |
| 변경 내용 비교 | <code>git diff</code>   | 수정된 내용 비교          |

| 목적         | 명령어                                       | 설명                |
|------------|-------------------------------------------|-------------------|
| 특정 파일 스테이징 | <code>git add &lt;file&gt;</code>         | 수정된 파일 스테이징       |
| 모든 변경 스테이징 | <code>git add .</code>                    | 전체 변경 스테이징        |
| 스테이징 해제    | <code>git reset &lt;file&gt;</code>       | add 취소 (작업 파일 유지) |
| 변경 취소      | <code>git checkout -- &lt;file&gt;</code> | 수정 전 상태로 되돌림      |

## 4 커밋(Commit) 관련

| 목적    | 명령어                                                     | 설명              |
|-------|---------------------------------------------------------|-----------------|
| 커밋 생성 | <code>git commit -m "메시지"</code>                        | 변경사항 기록         |
| 커밋 수정 | <code>git commit --amend</code>                         | 직전 커밋 메시지 수정    |
| 로그 확인 | <code>git log</code>                                    | 커밋 이력 확인        |
| 간단 로그 | <code>git log --oneline --graph --decorate --all</code> | 브랜치 구조 요약 보기    |
| 커밋 비교 | <code>git diff &lt;commit1&gt; &lt;commit2&gt;</code>   | 두 커밋 간 변경 내용 비교 |

## 5 브랜치 관리

| 목적         | 명령어                                         | 설명             |
|------------|---------------------------------------------|----------------|
| 브랜치 목록     | <code>git branch</code>                     | 로컬 브랜치 목록 표시   |
| 새 브랜치 생성   | <code>git branch &lt;name&gt;</code>        | 새로운 브랜치 생성     |
| 브랜치 이동     | <code>git checkout &lt;name&gt;</code>      | 해당 브랜치로 이동     |
| 생성과 동시에 이동 | <code>git checkout -b &lt;name&gt;</code>   | 브랜치 생성 후 바로 이동 |
| 브랜치 삭제     | <code>git branch -d &lt;name&gt;</code>     | 병합된 브랜치 삭제     |
| 강제 삭제      | <code>git branch -D &lt;name&gt;</code>     | 병합되지 않아도 삭제    |
| 원격 브랜치 목록  | <code>git branch -r</code>                  | 원격 브랜치 확인      |
| 브랜치 이름 변경  | <code>git branch -m &lt;new-name&gt;</code> | 현재 브랜치 이름 변경   |

## 6 병합(Merge) 및 리베이스(Rebase)

| 목적           | 명령어                                               | 설명                |
|--------------|---------------------------------------------------|-------------------|
| 브랜치 병합       | <code>git merge &lt;branch&gt;</code>             | 현재 브랜치에 다른 브랜치 병합 |
| 병합 중 충돌 발생 시 | <code>git status</code> / <code>git diff</code>   | 충돌 파일 확인          |
|              | <code>git diff --name-only --diff-filter=U</code> | 충돌난 파일 목록 확인      |

| 목적             | 명령어                                                         | 설명            |
|----------------|-------------------------------------------------------------|---------------|
|                | <code>git mergetool</code>                                  | 충돌 해결 도구 실행   |
| 충돌 해결 후 병합 완료  | <code>git add &lt;file&gt;</code> → <code>git commit</code> | 충돌 해결 후 커밋    |
| 리베이스 시작        | <code>git rebase &lt;branch&gt;</code>                      | 커밋 히스토리 재정렬   |
| 리베이스 중 충돌 발생 시 | <code>git rebase --continue</code>                          | 충돌 해결 후 계속 진행 |
| 리베이스 취소        | <code>git rebase --abort</code>                             | 리베이스 되돌리기     |

## 7 원격 저장소(Remote) 관련

| 목적                   | 명령어                                                  | 설명                 |
|----------------------|------------------------------------------------------|--------------------|
| 원격 저장소 추가            | <code>git remote add origin &lt;URL&gt;</code>       | 원격 저장소 등록          |
| 원격 저장소 확인            | <code>git remote -v</code>                           | 연결된 원격 저장소 목록      |
| 원격에서 가져오기            | <code>git fetch origin</code>                        | 원격 브랜치 업데이트        |
| pull (fetch + merge) | <code>git pull origin main</code>                    | 원격 변경사항 병합         |
| push                 | <code>git push origin main</code>                    | 로컬 커밋을 원격에 반영      |
| 새로운 브랜치 push         | <code>git push -u origin &lt;branch&gt;</code>       | 원격에 새 브랜치 업로드 및 연결 |
| 원격 브랜치 삭제            | <code>git push origin --delete &lt;branch&gt;</code> | 원격 브랜치 제거          |

## 8 변경 취소 및 복구

| 목적               | 명령어                                                      | 설명             |
|------------------|----------------------------------------------------------|----------------|
| 직전 커밋 취소 (파일 유지) | <code>git reset --soft HEAD~1</code>                     | 커밋만 되돌림        |
| 커밋 및 스테이징 취소     | <code>git reset --mixed HEAD~1</code>                    | add 및 커밋 모두 취소 |
| 완전 취소 (파일도 삭제)   | <code>git reset --hard HEAD~1</code>                     | 변경사항 완전 삭제     |
| 이전 커밋으로 되돌리기     | <code>git revert &lt;commit&gt;</code>                   | 해당 커밋 반대 작업 생성 |
| 특정 파일 복원         | <code>git checkout &lt;commit&gt; -- &lt;file&gt;</code> | 과거 커밋 버전으로 복원  |

## 9 임시 저장 (Stash)

| 목적          | 명령어                                  | 설명              |
|-------------|--------------------------------------|-----------------|
| 현재 작업 임시 저장 | <code>git stash</code>               | 변경사항 임시 보관      |
| 설명과 함께 저장   | <code>git stash push -m "메시지"</code> | 임시 저장에 설명 추가    |
| 임시 저장 목록    | <code>git stash list</code>          | 저장된 stash 목록 확인 |

| 목적          | 명령어                                    | 설명             |
|-------------|----------------------------------------|----------------|
| 최근 stash 적용 | <code>git stash apply</code>           | 가장 최근 stash 복원 |
| 특정 stash 적용 | <code>git stash apply stash@{2}</code> | 특정 stash 복원    |
| stash 삭제    | <code>git stash drop stash@{0}</code>  | 특정 stash 제거    |
| 전체 stash 삭제 | <code>git stash clear</code>           | 전부 제거          |

## 10 태그(Tag) 및 버전 관리

| 목적       | 명령어                                        | 설명            |
|----------|--------------------------------------------|---------------|
| 태그 생성    | <code>git tag v1.0.0</code>                | 특정 버전에 태그 붙이기 |
| 주석 포함 태그 | <code>git tag -a v1.0.0 -m "릴리스 설명"</code> | 주석 달린 태그 생성   |
| 태그 목록    | <code>git tag</code>                       | 전체 태그 확인      |
| 태그 푸시    | <code>git push origin v1.0.0</code>        | 특정 태그 푸시      |
| 전체 태그 푸시 | <code>git push origin --tags</code>        | 모든 태그 업로드     |

## 11 협업 및 리뷰 시 자주 쓰는 명령

| 목적             | 명령어                                        | 설명                   |
|----------------|--------------------------------------------|----------------------|
| 최신 main 반영     | <code>git pull origin main --rebase</code> | 내 브랜치에 최신 main 커밋 반영 |
| PR 전 확인용 차이 보기 | <code>git diff origin/main...HEAD</code>   | PR 전 로컬 변경점 확인       |
| 다른 사람 커밋 확인    | <code>git show &lt;commit-hash&gt;</code>  | 특정 커밋 상세 보기          |

## 12 실무 꿀팁

| 상황              | 명령어                                                  | 설명               |
|-----------------|------------------------------------------------------|------------------|
| 동일한 충돌 자동 해결 기억 | <code>git config --global rerere.enabled true</code> | 동일한 충돌 반복 해결 자동화 |
| 깔끔한 로그 보기       | <code>git log --graph --oneline --decorate</code>    | 트리 구조 한눈에 보기     |
| 최근 커밋별 변경 요약    | <code>git log --stat</code>                          | 커밋별 수정 파일/줄 수 요약 |
| 원격 삭제된 브랜치 정리   | <code>git fetch --prune</code>                       | 존재하지 않는 브랜치 정리   |

정리하자면

 **시험/면접**에서는 `init`, `add`, `commit`, `push`, `pull`, `merge`, `rebase`, `checkout`, `stash`, `reset`, `revert` 가 거의 90% 출제 포인트야.

👉 실무에서는 여기에 `fetch`, `rebase --continue`, `diff --staged`, `log --oneline`, `checkout -b`, `pull --rebase` 정도만 더 익히면 된다.

## ⚡ 충돌(Conflict) 해결 요약

1. `git pull` 시 충돌 발생 → 에디터에서 conflict 표시 확인.
2. 충돌난 파일에서 `<<<<<<`, `=====`, `>>>>>>` 부분 수동으로 수정 후

```
git add <파일명>
git commit
git push
```

3. 병합 완료!

## ✅ 시험 포인트

- “브랜치 병합 중 충돌이 발생했을 때 해결 절차”
- “원격 저장소에서 최신 코드를 가져오는 명령어” → `git pull`

## ◆ AI 프레임워크 (PyTorch)

딥러닝 개발에 가장 널리 쓰이는 프레임워크.

자동 미분(`autograd`), 모듈화된 모델 구성, GPU 가속이 핵심.

## 💡 기본 구조

```
import torch
import torch.nn as nn
import torch.optim as optim

# 1. 모델 정의
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc = nn.Linear(10, 1)
    def forward(self, x):
        return self.fc(x)
```

```
# 2. 학습 구성
model = Net()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# 3. 학습 루프
for epoch in range(10):
    inputs = torch.randn(16, 10)
    targets = torch.randn(16, 1)
    outputs = model(inputs)
    loss = criterion(outputs, targets)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

## 핵심 개념 정리

| 개념                | 설명                                                                        |
|-------------------|---------------------------------------------------------------------------|
| <b>Tensor</b>     | 다차원 배열 (NumPy 유사, GPU 계산 가능)                                              |
| <b>Autograd</b>   | 자동으로 미분값 계산 (역전파)                                                         |
| <b>Optimizer</b>  | SGD, Adam 등 파라미터 업데이트 알고리즘                                                |
| <b>DataLoader</b> | 배치 단위로 데이터 불러오기                                                           |
| <b>Device 설정</b>  | <code>torch.device("cuda" if torch.cuda.is_available() else "cpu")</code> |

### 시험 포인트

- "PyTorch에서 자동 미분을 담당하는 기능은?" → *Autograd*
- "모델의 가중치를 업데이트하는 구성요소는?" → *Optimizer*
- "GPU를 활용하기 위한 코드 구문은?" → `torch.device("cuda")`